
Solution

Annexure A – Scenario Based Assignment
Detailed Answers to All Five Questions (Q1 – Q5)

Course Name	Artificial Intelligence
Course Code	CSA17
Assessment	Assessment Tool 2 – Annexure A (Scenario Based Assignment)
Course Outcome	CO2 – Search & Game-Solving Techniques (BL3)
Weightage	50%
Duration	1 Hour

Declaration: I certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment, in accordance with the Code of Conduct in Annexure A.

Name / Reg. No.: _____

Signature of the Student: _____

Question 1: Emergency Drone Delivery in a Flood-Affected Region

Relevant Techniques: Greedy Best-First Search vs. A* Search

Scenario Recap: A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic — roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to a heuristic estimate (straight-line distance), partial real-time updates about blocked paths, and variable movement costs depending on terrain and weather. Due to urgency, the drone must quickly decide routes with minimum delay and risk.

i. Behaviour of Greedy Best-First Search under uncertainty

Greedy Best-First Search (GBFS) always expands the node whose heuristic value $h(n)$ — the straight-line distance to the hospital — appears smallest, completely ignoring the actual path cost $g(n)$ accumulated so far. In our simulation (`ai_scenario_solutions.py`, Scenario 1) GBFS greedily moves toward the node that looks nearest on the map, so it commits to routes such as Base to B to E to Hospital purely because each step reduces straight-line distance. Strengths: it is fast, expands very few nodes, and reacts quickly, which suits the drone's urgency. Weaknesses: because it never weighs the real terrain/weather cost, it can be lured toward a path that looks close on the map but is actually expensive or dangerous, it is not guaranteed to find the optimal (minimum-cost) route, and under uncertainty (sudden road blocks) it can be misled repeatedly since it has no mechanism to reconsider cost trade-offs — only proximity to the goal.

ii. How A balances cost and heuristic*

A* expands nodes based on $f(n) = g(n) + h(n)$, combining the real cost already travelled, $g(n)$, with the estimated remaining cost, $h(n)$. This balance means A* will happily take a slightly longer-looking detour if the actual accumulated cost keeps the total $f(n)$ lower, which is exactly what happened in our run: A* chose Base to B to D to Hospital with true cost 13, cheaper than the Greedy route's cost of 14, even though the Greedy route looked shorter heuristically at each step. Because the heuristic (straight-line distance) never overestimates the true remaining travel cost, i.e. it is admissible, A* is guaranteed to return the optimal path while still using the heuristic to prune away clearly unpromising directions, giving it far better solution quality than GBFS at only a modest increase in the number of nodes expanded.

iii. Impact of heuristic accuracy on both algorithms

Heuristic accuracy affects the two algorithms very differently. For GBFS, heuristic accuracy is everything — because cost is ignored entirely, a poor or misleading heuristic (e.g. straight-line distance across a lake with no bridge) sends the drone down a badly suboptimal or even unsafe corridor, with no correction mechanism. For A*, a more accurate (tighter) admissible heuristic mainly improves efficiency: it reduces the number of nodes expanded and speeds up the search, without ever sacrificing optimality, since A* still verifies real cost. If the heuristic is inadmissible (overestimates true cost), A* can lose its optimality guarantee, whereas GBFS was never optimal to begin with, so it has nothing further to lose. In short: for A*, better heuristics buy speed; for GBFS, better heuristics are the only thing standing between a reasonable route and a poor one.

iv. Effect of dynamic changes (blocked paths) on search performance

When a road becomes blocked mid-flight — modelled in our code by removing the D-to-Hospital edge after the drone has already committed to a plan — both algorithms must replan from the drone's current position using updated information. GBFS simply reruns its greedy proximity check and, because it never reasoned about cost, may re-select an equally naive detour (in our run it happened to still find Base-B-E-Hospital, cost 14). A* must re-open its frontier and re-evaluate $g(n)$ for all reachable nodes; in our run A* had to switch to Base-A-C-Hospital (cost 14) once the shorter D-Hospital leg was blocked,

expanding more nodes (7 versus 6) to guarantee this new route is still optimal given the obstruction. In general, dynamic blockages increase computational overhead for any real-time search agent and highlight the need for either incremental replanning (e.g., D*-Lite) or frequent, lightweight re-invocation of A* rather than a single static offline plan.

v. Recommended algorithm and justification

A* is the recommended algorithm for this emergency scenario. Although GBFS is marginally faster because it expands fewer nodes, the cost of a suboptimal or unsafe route in a medical-emergency context (extra minutes, extra fuel, entering higher-risk terrain) far outweighs the small computational saving. A* combines real path cost with the heuristic, guarantees the minimum-cost/minimum-risk route whenever the heuristic is admissible, and can be efficiently re-invoked (or replaced with an incremental variant such as D*-Lite) each time new blocked-path information arrives. Given the priorities stated in the scenario — minimum delay AND minimum risk, not just raw speed — A*'s balance of optimality and reasonable efficiency makes it the safer, more dependable choice for life-critical delivery under uncertainty.

Question 2: Smart City Traffic Signal Optimization

Relevant Techniques: Hill Climbing vs. Simulated Annealing / Genetic Algorithms

Scenario Recap: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize total waiting time, fuel consumption and traffic congestion. The system must continuously adjust signals based on traffic flow. However, the search space is extremely large, traffic patterns change dynamically, and the system may get stuck in locally optimal solutions.

Problem formulation and why traditional search is inefficient

This is best formulated as a continuous/combinatorial optimization problem rather than a path-finding problem: the state is a vector of signal-timing parameters for every intersection (green-light durations, phase offsets, cycle lengths), and the objective function combines total waiting time, fuel consumption and congestion into a single cost value to be minimised, as modelled by the synthetic `traffic_cost()` function in our implementation. Traditional uninformed or goal-based search (BFS/DFS/A*) is inefficient here because there is no single well-defined 'goal state' to search toward — instead every configuration has a quality score, the number of possible timing combinations across hundreds of intersections is combinatorially explosive, and traffic patterns change continuously, so any search that assumes a static, fully known environment (as classical path search does) would need to be re-run constantly and would still be far too slow. This calls for local, iterative-improvement (optimization) search rather than systematic path search.

i. Hill Climbing behaviour and its limitations

Hill Climbing starts from an initial timing configuration and repeatedly moves to the neighbouring configuration with the lowest cost, stopping as soon as no neighbour improves on the current state. In our simulation, Hill Climbing starting near `timing=22` immediately settles at `timing=20` (cost about 20.8) after only two steps because both of its immediate neighbours are worse — it has no way to see that a much better configuration exists further away. This is precisely its core limitation: it is a purely local, greedy technique with no memory or lookahead, so it is extremely likely to stop at the first local minimum it meets, it cannot detect flat regions or gentle slopes correctly, and its final answer depends heavily on the arbitrary starting configuration — unacceptable for a system meant to continuously reoptimise safely across an entire city.

ii. Local maxima, plateaus and ridges — real-world interpretation

A local maximum/minimum in this cost landscape corresponds to a set of signal timings that cannot be improved by any single small adjustment, even though a very different timing combination elsewhere would perform much better (e.g., synchronising a whole traffic corridor with 'green waves' rather than tuning one intersection at a time). A plateau corresponds to a stretch of near-identical timing configurations that all produce almost the same waiting time, so Hill Climbing wanders without clear improvement signal (real-world analogue: adjusting a signal by one second changes nothing noticeable because downstream congestion dominates). A ridge occurs when improvement is only possible by moving several correlated signals together, but Hill Climbing changes one variable at a time and each individual change looks unfavourable even though a combined change would help. All three traps are common in real traffic networks, where intersections are tightly coupled.

iii. How Simulated Annealing / Genetic Algorithms overcome these limitations

Simulated Annealing (SA) overcomes local optima by occasionally accepting a worse move with probability $\exp(-\Delta/T)$, where T (temperature) is gradually cooled over time; this lets it 'jump out' of local minima early in the search, while behaving more like Hill Climbing (and converging) as T

approaches zero. In our run, SA explored 300 candidate configurations and reached a lower-cost solution (about 20.6) than plain Hill Climbing, having sampled points far outside the basin Hill Climbing got stuck in. Genetic Algorithms overcome the same limitations differently — by maintaining an entire population of candidate timing plans, combining good partial solutions through crossover, and introducing random mutation, which naturally handles ridges (correlated variables can be inherited together) and plateaus (many candidates explore in parallel rather than one agent randomly wandering). Both methods trade raw speed for a much stronger ability to find near-global rather than merely local optima.

iv. Recommended approach and justification

For a city-scale system with hundreds of interacting, continuously-changing intersections, a Genetic Algorithm (or a hybrid GA plus local Hill Climbing/SA fine-tuning) is recommended for periodic, larger-scale re-optimisation, because its population-based, parallel search scales well to very large, high-dimensional search spaces and naturally supports distributed computation across intersections/zones. Simulated Annealing is recommended for fast, localised, near-real-time re-tuning of a single intersection or corridor as live traffic shifts, since it needs far less memory and converges quickly enough for continuous adjustment. Plain Hill Climbing alone is not recommended as the final production strategy — it is best used only as the final 'polishing' step after SA/GA has already found a strong region of the search space, combining scalability, adaptability and speed.

Question 3: Autonomous Mars Rover Exploration

Relevant Techniques: Online Search Agent under Partial Observability

Scenario Recap: An autonomous rover is deployed on Mars to explore unknown terrain. It must navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability, and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

i. Why an online search agent is required instead of offline search

Offline search algorithms (like standard A*) assume the entire state space — the full map, every hazard, every terrain cost — is known in advance, so a complete solution path can be computed once before any action is taken. The rover cannot satisfy this assumption: craters and rocks are only discovered once they are within sensor range, so a full map simply does not exist beforehand. An online search agent instead interleaves computation and action — it computes the best next move (or a short plan) using only currently known information, executes it, senses the result, and re-plans. Our implementation demonstrates this with a local A* replanning loop that repeatedly plans a route to the goal while avoiding only the hazards discovered so far, halting and replanning the instant a new hazard is sensed — this 'sense, plan a little, act, repeat' cycle is the defining trait of online search and is the only workable strategy when the map is unknown or only partially known in advance.

ii. Challenges of partial observability and dynamic environments

Partial observability means the rover's decisions must be based on incomplete, locally sensed information, so a route that appears safe and efficient from current knowledge can turn out to be blocked one step later — in our simulation the rover triggered three separate re-plans out of six actual hazards in the terrain, discovering only two of them along its travelled path. This creates several concrete challenges: the rover can be led into a 'dead end' requiring costly backtracking since some online strategies cannot guarantee optimal exploration order; every new piece of sensed information can invalidate the current plan, forcing frequent, computationally expensive re-planning; and if the environment is also dynamic (e.g., dust storms shifting loose rocks), even previously 'safe' cells may need to be re-verified, so the rover can never fully trust old knowledge and must balance caution against making forward progress within limited power and time budgets.

iii. How the rover builds and updates its internal model

The rover maintains an internal knowledge base — in our implementation, a set of 'known hazard' cells — that starts empty and grows only from direct sensor observations as the rover moves. Before each move it re-plans a route using `local_a_star()` against this current belief state, treating unexplored cells as optimistically traversable (a common online-search assumption is that the world is 'freely explorable' until proven otherwise). The moment the planned next cell is sensed to be hazardous, that cell is permanently added to the known-hazard set (the rover's model is updated), the current plan is discarded, and a fresh plan is computed from the rover's present position using the improved model. Over time this internal model becomes a progressively more accurate partial map, letting later plans avoid regions that earlier plans had to discover the hard way.

iv. Online search compared with uninformed and informed offline search

Uninformed search (e.g., BFS/DFS/Uniform-Cost Search) explores blindly without any estimate of distance to the goal and assumes complete map knowledge; it is not viable for the rover since it cannot even be executed without a known map. Informed offline search (classical A*) is more efficient than uninformed search because it uses a heuristic, but it still assumes the whole environment, including hazard locations, is known before the first move — an assumption the rover cannot meet. Online

search sacrifices the completeness/optimality guarantees of offline A* (a single re-plan sequence is not guaranteed to be the globally shortest possible path, since some distance is 'wasted' discovering hazards) in exchange for being executable at all under partial observability; it essentially runs many small, informed (heuristic-guided) local searches back-to-back, inheriting A*'s efficiency benefits within each individual re-plan while remaining robust to an incomplete world model.

v. Justification of the most suitable approach

An online, informed search strategy — local A* (or the closely related LRTA*, Learning Real-Time A*, which additionally remembers updated heuristic values between visits to a cell) — is the most suitable approach. It directly addresses the three defining constraints of the scenario: uncertainty, because it never assumes more knowledge than the rover actually has and safely re-plans around newly discovered hazards; adaptability, because it naturally incorporates every new sensor reading into future decisions without needing a restart; and safety, because hazards are avoided the moment they are detected rather than being 'baked in' to a stale offline plan. LRTA*'s added heuristic-learning also means the rover's future re-plans get progressively cheaper as it accumulates experience of the terrain, an important efficiency gain over many independent missions.

Question 4: University Exam Timetable Generation

Relevant Techniques: Constraint Satisfaction Problem — Backtracking with Forward Checking

Scenario Recap: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: no student should have overlapping exams, a limited number of exam halls exist, certain subjects must be scheduled before others, and faculty availability and special student accommodations must be respected. The complexity increases as the number of courses and students grows.

CSP formulation

The exam-timetabling problem maps naturally onto the CSP framework: Variables are the individual courses to be scheduled (in our implementation: AI, DBMS, OS, CN, ML, SE). Domains are the set of available exam slots for each course (Day1-9AM, Day1-1PM, Day2-9AM, Day2-1PM) — in a real deployment this domain would further be restricted per course by faculty availability and any special accommodation requirements. Constraints fall into three families: (1) binary conflict constraints — two courses that share enrolled students cannot be assigned the same slot (modelled as a CONFLICTS set of course pairs), (2) resource/hall constraints — no more than HALL_CAPACITY courses may be scheduled in the same slot since halls are limited, and (3) precedence constraints — certain subjects must be scheduled before others (e.g., DBMS before OS), enforced by comparing slot ordering. A correct assignment maps every course to exactly one slot such that all three constraint families are satisfied simultaneously.

i. Variables, domains and constraints — detailed definition

Formally: $X = \{AI, DBMS, OS, CN, ML, SE\}$ (the six courses). $D(x) = \{Day1-9AM, Day1-1PM, Day2-9AM, Day2-1PM\}$ for every course x , i.e., the four candidate exam slots (in practice this domain is narrowed per-course by faculty and accommodation constraints before search even begins). C = the constraint set, comprising: (a) binary 'not-equal' constraints between any two courses known to share students, e.g., AI and ML cannot share a slot; (b) a global cardinality constraint on each slot, capping simultaneous exams at HALL_CAPACITY (2 in our model) to reflect limited exam-hall availability; and (c) ordering constraints of the form slot(before) earlier-than slot(after) for precedence pairs like (DBMS, OS). A solution is a complete, consistent assignment of a single slot to every course that violates none of these constraints.

ii. How backtracking search works in this scenario

Backtracking search assigns courses one at a time (in our implementation, selecting the most-constrained course — the one with the smallest remaining domain — first, a standard minimum-remaining-values heuristic), tentatively picking a slot from its domain and checking it against every constraint involving already-assigned courses. If the tentative slot is consistent, the search recurses to assign the next course; if it violates a conflict, capacity, or precedence constraint, that slot is rejected and the next candidate slot in the domain is tried. When every slot for a course fails, the algorithm backtracks — it undoes the previous course's assignment and tries an alternative for it instead. This systematic trial-and-undo process continues until either a fully consistent timetable is found (as in our run, which produced a complete valid timetable) or the search space is exhausted.

iii. How forward checking improves efficiency

Plain backtracking only discovers a conflict once it directly tries an inconsistent assignment, which can waste significant search effort deep in the tree. Forward checking improves on this by, immediately after each new assignment, removing now-inconsistent values from the domains of all not-yet-assigned, related courses — for example, once AI is placed in Day1-9AM, forward checking

immediately removes Day1-9AM from ML and DBMS's remaining domains (since they conflict with AI) rather than waiting to discover this later. If forward checking ever empties a domain completely (a 'domain wipeout'), the search backtracks immediately without wasting time exploring that doomed branch further. In our implementation this let the search find a complete valid timetable while exploring only 6 nodes total, a substantial reduction compared to what plain backtracking without pruning would typically need to explore as the number of courses grows.

iv. Real-world challenges and justification of the best strategy for scalability

In practice, timetabling grows harder as more courses, students, halls and faculty constraints are added: the number of pairwise conflict constraints can grow roughly quadratically with course count, hall/room capacity constraints add global (not just binary) restrictions, and 'soft' constraints — student accommodation preferences, faculty preferred slots — often need to be handled as costs to minimise rather than hard rules, turning the problem into a constrained optimisation problem rather than a pure CSP. Backtracking with forward checking (optionally strengthened with arc-consistency techniques such as AC-3, and variable/value-ordering heuristics like minimum-remaining-values and least-constraining-value) is recommended as the core scalable strategy, because it prunes the search space early rather than after the fact; for very large universities this is often combined with local-search repair methods (e.g., min-conflicts) that quickly fix a small number of remaining violations in an otherwise-complete timetable, giving both correctness and practical scalability.

Question 5: Strategic Game AI for Real-Time Decision Making

Relevant Techniques: Minimax and Alpha-Beta Pruning

Scenario Recap: A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, make decisions within strict time limits, and evaluate complex game states with multiple possible moves. The decision tree is extremely large, making computation expensive.

i. How Minimax models this problem

Minimax treats the game as a two-player, zero-sum contest between a maximising player (the AI) and a minimising player (the opponent), building a game tree in which each level alternates whose turn it is to move. At each leaf (terminal or depth-limited) node, an evaluation function scores the position from the AI's perspective; the algorithm then propagates these scores back up the tree, with the maximising player always choosing the child of highest value and the minimising player always choosing the child of lowest value, under the assumption that the opponent plays optimally against the AI. In our Tic-Tac-Toe demonstration (a minimal but complete zero-sum game used to make the full tree fully verifiable) Minimax correctly identified a move with a guaranteed-safe outcome, value 0 (a draw against optimal opposition), after exploring the entire remaining game tree of 185 nodes — the same principle scales directly to any larger RTS decision tree, only with a vastly larger branching factor and depth.

ii. Computational challenges of large game trees

The central challenge is combinatorial explosion: the size of a full game tree grows exponentially with depth, roughly b^d for branching factor b and depth d , so real-time strategy games — which can have dozens of possible unit actions per turn and long time horizons — produce trees far too large to search exhaustively within any practical time or memory budget. Plain Minimax, as implemented in our code, must visit every single node down to a terminal state (185 nodes even for the small Tic-Tac-Toe example), which becomes utterly infeasible for RTS-scale branching factors. This forces trade-offs: the search must be depth-limited using a heuristic evaluation function at non-terminal cut-off nodes, and wasted computation on clearly inferior branches must be eliminated wherever possible, motivating techniques like alpha-beta pruning, move ordering, and transposition tables.

iii. How Alpha-Beta pruning improves efficiency

Alpha-Beta pruning augments Minimax with two bounds — alpha, the best value the maximiser can currently guarantee, and beta, the best value the minimiser can currently guarantee — and cuts off the exploration of any branch that cannot possibly influence the final decision: once $\beta \leq \alpha$ at a node, the remaining siblings are skipped because a rational opponent would never allow play to reach that branch. Crucially, alpha-beta pruning returns exactly the same result as plain Minimax; it only removes provably irrelevant computation. In our implementation this reduced nodes explored on the identical board position from 185 down to 114, a 38.4% cut in search effort, and the benefit grows dramatically with tree size and, especially, with good move ordering (searching the most promising moves first maximises the number of cutoffs, and can reduce an effective branching factor from b to roughly its square root under ideal ordering).

iv. Designing an evaluation function and justifying the chosen factors

Because real RTS trees cannot be searched to a terminal state, the AI must cut the search off early and score the resulting non-terminal position using a heuristic evaluation function. For our demonstration game the evaluation only needed to detect a win/loss/draw (+10, -10, 0) since the whole tree was searchable; for a genuine RTS position, a practical evaluation function would combine several weighted factors: material/unit-strength balance (unit count and quality on each side), positional/territorial control

(map area, resource-node ownership), tactical threat level (units in immediate danger or about to deal damage), and economic factors (resource-gathering rate, production capacity). These factors are chosen because they are each cheap to compute (essential when millions of nodes are evaluated) while collectively correlating strongly with long-run winning chances, letting the search stay shallow yet still make strategically sound decisions.

v. Recommended strategy for balancing optimality and real-time performance

The recommended strategy is Alpha-Beta pruning combined with iterative deepening (search to depth 1, then 2, then 3, and so on, always keeping the best move found so far, so a legal move is available the instant the time budget expires), strong move ordering (trying previously-good or heuristically promising moves first to maximise pruning), and a well-tuned, cheap-to-compute evaluation function at the cutoff depth. This combination gives up strict optimality — the AI is no longer guaranteed to find the single best move in the full, intractably large game tree — in exchange for bounded, predictable computation time and consistently strong decisions, which is precisely the trade-off a real-time strategy game opponent needs: it must always respond within the strict time limit while still playing at a genuinely competitive level against human opponents.

Note on Implementation

All algorithms discussed above were additionally implemented and empirically verified in **ai_scenario_solutions.py** (submitted alongside this document). Where a specific numeric result is quoted in an answer (e.g., path cost, nodes expanded, percentage reduction), that figure was produced directly by running the accompanying code, and the full execution trace plus a consolidated visual summary are provided in **output.png** and **Report.pdf**.